

0.85.1. ALPHABET_GRAS — les lettres en gras

`#include <ALPHABET_GRAS>` : `ALPHABET_GRAS[0]` est le A en gras, à côté de l'alphabet ordinaire.

1. Ce qu'il faut comprendre

Ce que c'est : `ALPHABET_GRAS[0]` est un **A en gras**, `ALPHABET_GRAS[25]` un Z en gras. Ce sont 26 tuiles dessinées, ajoutées à côté de la police : l'alphabet ordinaire ne change pas.

La ligne à écrire : `#include <ALPHABET_GRAS>`.

Ce qu'il coûte : **beaucoup**, environ **1 200 octets** dès la première lettre : les 26 dessins et le tableau qui les range partent ensemble, même pour un seul A.

Essaie : pose le A normal à côté du A gras, pour comparer.

On retrouve ALPHABET_GRAS dans 3 leçons

[0.86](#), [0.86.1](#), [0.86.2](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1304 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
main()         25 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
CopierTuiles   18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         1120 o ← CopierTuiles – la police entière, et 26 tuiles dessinées
ALPHABET_GRAS_0 26 o ← poser() – une table gravée
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser() × 2    18 o sur place, dans main() + ALPHABET_GRAS_0 (26 o)
image() × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <ALPHABET_GRAS> : les lettres en gras ----

#include <ALPHABET_GRAS> // ALPHABET_GRAS : 26 lettres grasses
#include <ALPHABET>
#include <poser>
```

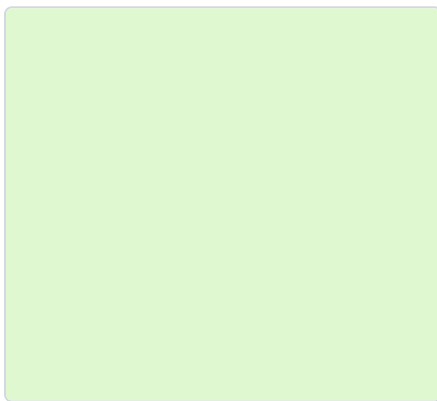
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(8, 8, ALPHABET_GRAS[0]);    // le A gras
    poser(10, 8, ALPHABET[0]);        // le A ordinaire, pour comparer

    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un A en gras, et un A ordinaire à sa droite.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ une tuile est posée en (8, 8)
- ✓ ce n'est pas le A ordinaire (44 et 1)
- ✓ le A ordinaire est là

Ensuite : **0.86. L'alphabet en gras, avec poserS** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++